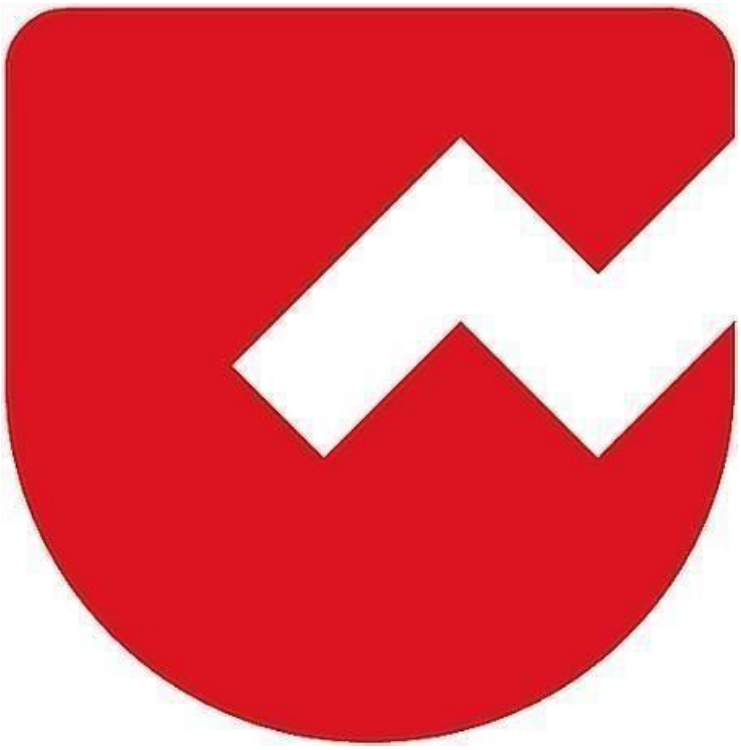
Zoho Schools for Graduate Studies



Notes

Multithreading

Threads:

- A **thread** is a lightweight sub-process that represents an independent path of execution within a program.
- In Java, every program has **at least one thread**, called the **main thread**, which starts automatically when the program begins execution.
- Threads allow multiple tasks to run **concurrently**, improving performance and responsiveness.

Types of Threads:

Main Thread (Default Thread)

- Created automatically when the Java program starts.
- Responsible for executing the main() method.

Child Threads (User-defined threads)

- Created by the programmer to run tasks parallelly with the main thread.

Defining a Thread in Java:

A thread is defined by overriding the run ( ) method.

run() Method

- Contains the logic that the thread will execute.
- When a thread starts, only the run ( ) method code runs.

How to create a Thread in Java:

There are two ways:

1. Extending the Thread class

2. Implementing the Runnable interface

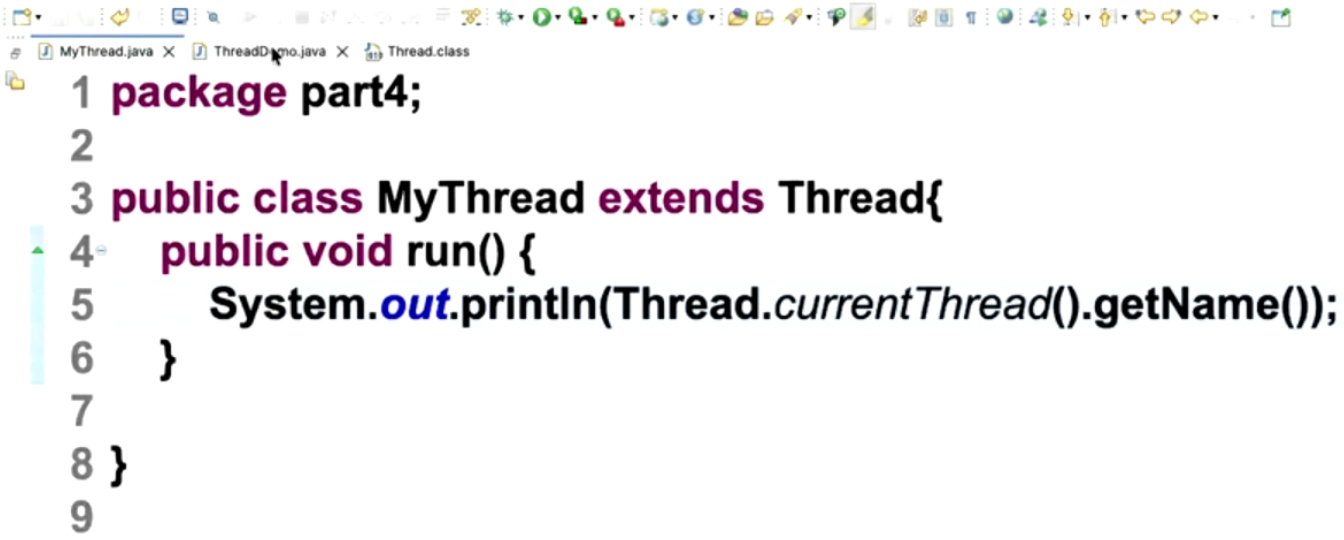
Method 1 – Extending the Thread Class:

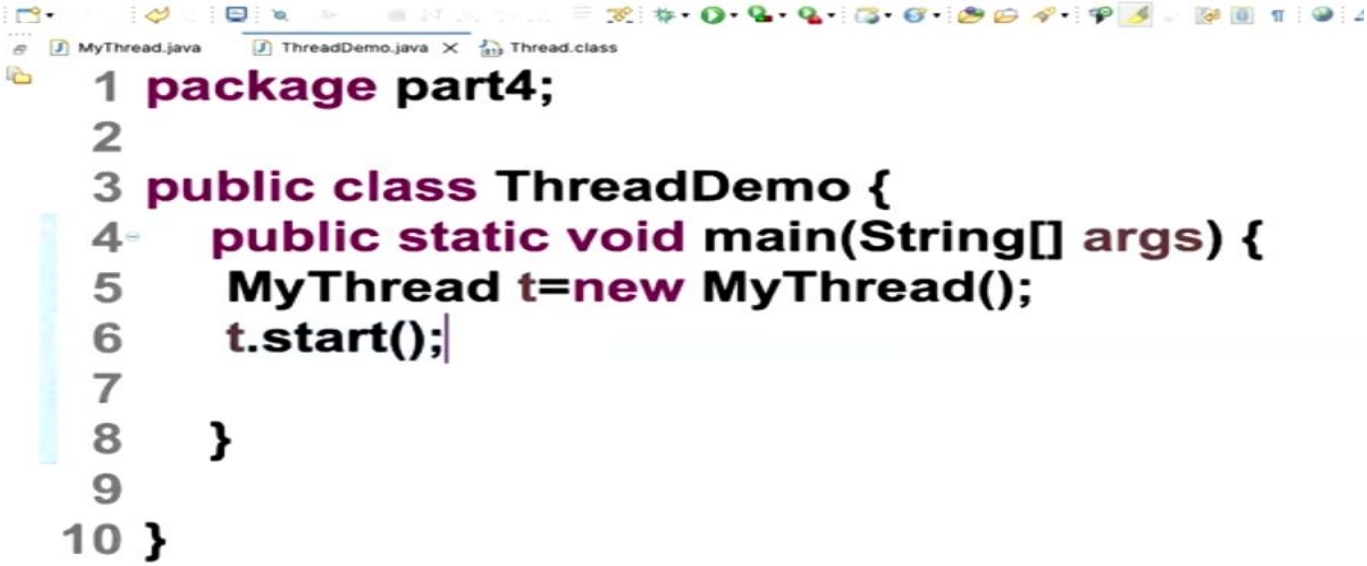
Steps

1. Create a class that extends Thread.
2. Override the run() method.
3. Create an object of your class.
4. Call the start() method → this internally calls run().

Important Notes

- You must use **start()** (NOT run()) to start a new thread.
- A thread **executes only once**. Restarting a thread causes: **→IllegalThreadStateException.**





Method 2 – Implementing Runnable Interface:

Why Runnable is better?

- Java supports multiple inheritance only through interfaces.
- To extend another class and still create a thread → Runnable is recommended.
- More flexible for large applications.

Steps

1. Create a class that implements Runnable.
2. Override run().
3. Create a Thread object and pass your Runnable object to it.
4. Call Start().

*MyThread.java ThreadDemo.java Thread.class NewThread.java

```java
package part4;

public class NewThread implements Runnable{
    public void run() {
        for(int i=1;i<=5;i++)
        System.out.println(Thread.currentThread().getName());
    }

}

```

MyThread.java ThreadDemo.java Thread.class NewThread.java

```java
package part4;

public class ThreadDemo {
    public static void main(String[] args) {
    System.out.println(Thread.currentThread().getName() + "  Begins!");

    Thread t1=new Thread(new NewThread());
     t1.setName("First");
     t1.start();

     Thread t2=new Thread(new NewThread());
     t2.setName("\t\t\tSecond");
     t2.start();

```

Thread Scheduler:

The Thread Scheduler in Java is responsible for deciding the execution order of threads. It uses algorithms like preemptive scheduling and time slicing to determine which thread gets CPU time. Thread execution order is never guaranteed.

Preemptive Scheduling:

Forcible deallocation of a resource from a thread is termed as Preemption

Non-Preemptive:

Thread – A piece of code which has its own path of execution

Sequential Execution:

Tasks run one after another, in a fixed order. Only one task is running at any moment.

Concurrent Execution:

Multiple tasks overlap in time. Not necessarily at the exact same instant but it looks like they run together.

Parallel Execution:

Multiple tasks run at the exact same time, using multiple CPU cores. So actual simultaneous execution happens.